

Project: Confronto tra regole di voto su profili casuali

scelta collettiva, preferenze eterogenee e metodi computazionali

1. Obiettivi della dispensa

Questa dispensa introduce i sistemi di voto come caso di studio per un corso di metodi computazionali per modelli stocastici. Il problema centrale e' capire come diverse regole di aggregazione trasformino un insieme di preferenze individuali in una decisione collettiva.

Gli obiettivi sono cinque:

1. formalizzare il profilo delle preferenze di una popolazione di elettori;
2. introdurre diverse regole di voto e confrontarne gli esiti;
3. mostrare come la generazione casuale dei profili produca una distribuzione di risultati collettivi;
4. identificare osservabili quantitative per confrontare robustezza, paradossi e sensibilita' istituzionale;
5. usare il problema come base per attivita' di laboratorio e simulazione Monte Carlo.

Dal punto di vista didattico, questo modulo e' importante perche' mostra in modo molto chiaro che, anche a preferenze individuali fissate, il risultato collettivo dipende dalla regola di aggregazione. La procedura non e' un dettaglio tecnico: e' parte integrante del fenomeno studiato.

2. Motivazione generale

I sistemi di voto sono uno dei luoghi classici in cui matematica, informatica, scienze sociali, economia e teoria delle decisioni si incontrano in modo naturale.

In termini astratti, il problema e' semplice: una popolazione di elettori deve scegliere tra piu' alternative. Tuttavia, appena il numero dei candidati supera due, compaiono questioni profonde:

- regole diverse possono selezionare vincitori diversi;

- la preferenza collettiva può risultare ciclica anche quando le preferenze individuali sono perfettamente ordinate;
- il risultato può essere molto sensibile all'ingresso o all'uscita di un candidato;
- piccole perturbazioni del profilo possono modificare il vincitore.

Questo rende i sistemi di voto un case study particolarmente efficace per il corso. Il problema è concettualmente accessibile, ma ricco di fenomeni non banali. Inoltre, si presta molto bene a una impostazione computazionale basata su generazione casuale di profili, analisi statistica degli esiti e confronto tra istituzioni alternative.

3. Definizione formale del problema

3.1 Elettori, candidati e profili di preferenza

Consideriamo una popolazione di N elettori e un insieme di m candidati, con $m \geq 2$.

Ogni elettore i possiede una classifica completa dei candidati, che rappresentiamo come una permutazione

$$\pi_i = (\pi_i(1), \dots, \pi_i(m)),$$

dove:

- $\pi_i(1)$ è il candidato preferito da i ;
- $\pi_i(m)$ è il candidato meno preferito.

L'intero stato del sistema è il profilo delle preferenze:

$$\Pi = (\pi_1, \dots, \pi_N).$$

Il profilo Π è dunque l'oggetto fondamentale del modello. Tutte le regole di voto che considereremo prendono in input un profilo e restituiscono uno o più vincitori.

3.2 Matrice di rango

Per scopi computazionali, può essere utile rappresentare il profilo anche tramite una matrice dei ranghi. Indichiamo con

$$R_{ia}$$

la posizione del candidato a nella classifica dell'elettore i . Valori piccoli corrispondono a candidati più apprezzati.

Questa rappresentazione e' molto comoda per implementare punteggi, confronti a coppie e misure di distanza tra profili.

3.3 Preferenze come variabili casuali

La componente stocastica del progetto non consiste in una dinamica temporale del voto, ma nella generazione casuale dei profili. Il profilo Π viene trattato come una variabile aleatoria estratta da una certa distribuzione su tutte le classifiche possibili.

Questa osservazione e' metodologicamente importante. In molti modelli del corso il rumore entra nella dinamica temporale del sistema. Qui, invece, la stocasticita' entra nella distribuzione delle preferenze iniziali della popolazione.

4. Famiglie di profili casuali

Per confrontare le regole di voto in modo sistematico conviene introdurre alcune famiglie di profili generati casualmente.

4.1 Impartial culture

Nel caso piu' semplice, ogni elettore estrae la propria classifica in modo uniforme tra tutte le $m!$ permutazioni possibili.

Formalmente, le preferenze individuali π_i sono indipendenti e identicamente distribuite con legge uniforme sul gruppo delle permutazioni.

Questo e' il benchmark piu' neutrale e piu' semplice da simulare.

4.2 Modello spaziale delle preferenze

Una seconda possibilita' consiste nel collocare elettori e candidati in uno spazio ideologico, per esempio una linea o un piano.

Ogni elettore i ha una posizione $x_i \in \mathbb{R}^d$ e ogni candidato a una posizione $y_a \in \mathbb{R}^d$.

Le preferenze dell'elettore derivano dalla distanza:

$$u_i(a) = -\|x_i - y_a\|,$$

oppure, piu' in generale, da una utilita' rumorosa

$$u_i(a) = -\|x_i - y_a\| + \sigma \eta_{ia},$$

dove η_{ia} e' uno shock casuale.

La classifica π_i si ottiene ordinando i candidati per utilita' decrescente.

Questo schema introduce una struttura geometrica nelle preferenze e permette di studiare la relazione tra regole di voto e prossimita' spaziale.

4.3 Profili con blocchi o fazioni

Per introdurre correlazioni tra elettori si puo' supporre che la popolazione sia suddivisa in gruppi. Ogni gruppo condivide una distribuzione di preferenze simile, pur con una certa variabilita' interna.

Per esempio, si possono definire G blocchi, ciascuno caratterizzato da un ordine centrale, e generare ogni classifica individuale come una perturbazione casuale dell'ordine del proprio gruppo.

Questo schema e' utile per studiare polarizzazione, voto di fazione e frammentazione.

5. Regole di voto considerate

5.1 Plurality

Nella regola plurality ogni elettore assegna un voto al proprio primo classificato. Vince il candidato con il maggior numero di primi posti.

Se indichiamo con

$$V_a^{\text{plu}} = \sum_{i=1}^N \mathbf{1}\{\pi_i(1) = a\},$$

allora il vincitore e' il candidato con valore massimo di V_a^{plu} .

Questa regola e' molto semplice, ma puo' penalizzare candidati ampiamente accettabili che raramente compaiono al primo posto.

5.2 Majority runoff

Nel sistema a doppio turno si selezionano prima i due candidati con piu' primi posti. Nel secondo turno, tra questi due, vince quello che batte l'altro nel confronto testa a testa.

Questa regola e' interessante perche' combina una selezione iniziale per intensita' del consenso e una selezione finale per maggioranza binaria.

5.3 Borda count

Nel metodo di Borda, se vi sono m candidati, un candidato riceve $m - 1$ punti quando e' primo, $m - 2$ quando e' secondo, e cosi' via fino a 0 punti quando e' ultimo.

Il punteggio del candidato a e'

$$V_a^{\text{Bor}} = \sum_{i=1}^N (m - R_{ia}).$$

Vince il candidato con punteggio totale massimo.

Questa regola valorizza l'accettabilita' media e non solo i primi posti.

5.4 Approval voting

Per definire l'approval voting serve una regola che traduca una classifica in un insieme di candidati approvati.

La scelta piu' semplice, per una dispensa introduttiva, e' assumere che ogni elettore approvi i primi k candidati della propria classifica, con $1 \leq k < m$.

Il punteggio del candidato a e' allora

$$V_a^{\text{app}} = \sum_{i=1}^N \mathbf{1}\{R_{ia} \leq k\}.$$

Vince il candidato con il maggior numero di approvazioni.

Questa formalizzazione e' molto utile in laboratorio, perche' consente di confrontare in modo trasparente plurality e approval variando il parametro k .

5.5 Confronti a due a due e criterio di Condorcet

Per ogni coppia di candidati (a, b) definiamo

$$M_{ab} = \sum_{i=1}^N \mathbf{1}\{a \succ_i b\},$$

dove $a \succ_i b$ significa che l'elettore i preferisce a a b .

Il candidato a e' un vincitore di Condorcet se, per ogni altro candidato $b \neq a$,

$$M_{ab} > M_{ba}.$$

Un ciclo di Condorcet si verifica quando le preferenze collettive a coppie non sono transitive. Per esempio, possono verificarsi situazioni del tipo

$$a \succ b, \quad b \succ c, \quad c \succ a$$

a livello di maggioranza.

6. Osservabili da misurare

Per trasformare il problema in un case study computazionale conviene definire alcune osservabili aggregate.

6.1 Frequenza di disaccordo tra regole

Una osservabile naturale e' la probabilita' che due regole diverse selezionino vincitori diversi:

$$P_{\text{div}} = \Pr(W^{(1)} \neq W^{(2)}).$$

Questa misura quantifica la sensibilita' istituzionale dell'esito.

6.2 Probabilita' di esistenza di un vincitore di Condorcet

Definiamo

$$P_{\text{Con}} = \Pr(\text{esiste un vincitore di Condorcet}).$$

Complementarmente, si puo' misurare la probabilita' di cicli di maggioranza.

6.3 Frequenza di ciclo di Condorcet

Definiamo

$$P_{\text{cyc}} = \Pr(\text{la relazione di maggioranza contiene un ciclo}).$$

Questa osservabile e' particolarmente importante per mostrare che aggregare preferenze razionali individuali non garantisce razionalita' collettiva.

6.4 Robustezza del vincitore a perturbazioni locali

Dato un profilo Π , si puo' perturbare casualmente una piccola frazione di classifiche o scambiare due candidati in alcune schede e osservare se il vincitore cambia.

Una possibile misura di robustezza e'

$$R = \Pr(W(\Pi') = W(\Pi)),$$

dove Π' e' un profilo perturbato.

6.5 Distanza spaziale dal votante mediano

Nel modello spaziale, se gli elettori e i candidati sono collocati su una linea, si puo' misurare la distanza tra il vincitore e la posizione mediana degli elettori.

Questa misura e' utile per confrontare regole orientate ai primi posti con regole piu' centrate sul consenso diffuso.

6.6 Sensibilita' al numero di candidati

Si puo' studiare come cambiano le osservabili al variare di m . In particolare, e' interessante verificare se l'aumento del numero di candidati accresca:

- la frequenza di divergenza tra regole;
- la probabilita' di cicli;
- la instabilita' del vincitore.

7. Domande scientifiche che il modello permette di studiare

Il progetto consente di affrontare alcune domande molto naturali.

1. Quanto spesso regole di voto diverse producono vincitori diversi sugli stessi profili?
2. Come dipende la probabilita' di ciclo di Condorcet dal numero di elettori e candidati?
3. Le regole basate sui primi posti differiscono sistematicamente da quelle che valorizzano il consenso diffuso?
4. Nei modelli spaziali, quali regole selezionano candidati piu' vicini al centro della distribuzione degli elettori?
5. In presenza di blocchi o fazioni, alcune regole risultano piu' stabili o meno sensibili alla frammentazione?
6. Quanto conta la specificazione del modello generativo delle preferenze nel determinare gli esiti aggregati?

8. Pseudocodice generale

Di seguito una struttura semplice per una simulazione Monte Carlo.

8.1 Input

- numero di elettori N
- numero di candidati m
- numero di simulazioni S
- modello di generazione delle preferenze
- eventuali parametri del modello generativo

- scelta del parametro k per approval voting

8.2 Pseudocodice

Per $s = 1, \dots, S$:

1. genera un profilo casuale $\Pi^{(s)}$;
2. calcola il vincitore plurality;
3. calcola il vincitore Borda;
4. calcola il risultato majority runoff;
5. calcola il vincitore approval;
6. costruisci la matrice dei confronti a coppie M ;
7. verifica se esiste un vincitore di Condorcet;
8. verifica se sono presenti cicli di maggioranza;
9. salva:
 - i vincitori delle diverse regole;
 - le osservabili binarie o numeriche del profilo;
 - eventuali misure di robustezza;
 - eventuali distanze spaziali.

Alla fine:

1. calcola le frequenze empiriche delle diverse osservabili;
2. confronta gli esiti al variare dei parametri;
3. visualizza i risultati con tabelle, istogrammi o heatmap.

9. Pseudocodice di alcune sottoprocedure

9.1 Generazione di un profilo in impartial culture

Per ogni elettore i :

1. genera una permutazione casuale dei candidati;
2. assegna la permutazione a π_i .

Restituisci il profilo Π .

9.2 Costruzione della matrice di confronti a due a due

Per ogni coppia ordinata (a, b) con $a \neq b$:

1. inizializza $M_{ab} = 0$;
2. per ogni elettore i :
 - se a precede b nella classifica π_i , incrementa M_{ab} di una unita'.

Alla fine, la matrice M contiene tutti i confronti a coppie.

9.3 Calcolo del vincitore di Borda

1. inizializza tutti i punteggi a zero;
2. per ogni elettore i :
 - per ogni posizione $r = 1, \dots, m$:
 - assegna $m - r$ punti al candidato in posizione r ;
3. restituisci il candidato con punteggio massimo.

9.4 Test di robustezza locale

Dato un profilo Π :

1. calcola il vincitore iniziale W ;
2. genera una versione perturbata Π' modificando una piccola quota di schede;
3. ricalcola il vincitore W' ;
4. registra se $W' = W$;
5. ripeti molte volte per stimare la robustezza empirica.

10. Commento didattico sul pseudocodice

Dal punto di vista computazionale, questo progetto permette di esercitare competenze molto importanti:

- rappresentazione di dati combinatori;
- generazione di variabili casuali discrete;
- uso di simulazioni Monte Carlo;
- confronto tra algoritmi alternativi sullo stesso input;
- definizione e stima di osservabili statistiche;
- interpretazione del rapporto tra regole microscopiche e risultati aggregati.

Il progetto ha anche un pregio didattico ulteriore: pur essendo relativamente semplice da implementare, produce risultati concettualmente molto ricchi. In questo senso e' ideale per studenti provenienti da discipline diverse.

11. Schema del laboratorio

11.1 Laboratorio 1 - Implementazione delle regole di voto

Obiettivo

Implementare plurality, Borda, majority runoff e approval voting, e verificare che possano produrre vincitori diversi sullo stesso profilo.

Attività

1. fissare valori iniziali, ad esempio

$$N = 100, \quad m = 4;$$

2. generare profili casuali in impartial culture;
3. implementare le diverse regole;
4. confrontare i vincitori su molti profili;
5. costruire esempi in cui i risultati divergono.

Domande guida

- quanto spesso plurality e Borda selezionano lo stesso candidato?
- approval voting e' piu' vicino a plurality oppure a Borda?
- il runoff corregge sistematicamente il risultato di plurality oppure no?

Output richiesto

- codice sorgente;
- esempi di profili con esiti differenti;
- tabelle di frequenza dei vincitori;
- breve commento interpretativo.

11.2 Laboratorio 2 - Cicli di Condorcet

Obiettivo

Studiare la probabilita' di cicli di maggioranza al variare del numero di elettori e candidati.

Attivita'

1. costruire la matrice dei confronti a due a due;
2. rilevare la presenza di un vincitore di Condorcet o di un ciclo;
3. ripetere l'esperimento per diversi valori di N e m ;
4. stimare empiricamente P_{Con} e P_{cyc} .

Domande guida

- la probabilita' di ciclo cresce con il numero di candidati?
- il numero di elettori riduce o amplifica l'instabilita'?
- un vincitore di Condorcet, quando esiste, coincide spesso con il vincitore di altre regole?

Output richiesto

- grafici o tabelle delle frequenze;
- confronto tra casi con $m = 3, 4, 5$;
- commento sul significato del paradosso.

11.3 Laboratorio 3 - Preferenze spaziali

Obiettivo

Confrontare le regole di voto quando le preferenze sono generate da una distanza in uno spazio ideologico.

Attività

1. collocare elettori e candidati su una linea o in un piano;
2. derivare le classifiche dalla distanza o dall'utilità rumorosa;
3. applicare le diverse regole di voto;
4. misurare la distanza tra vincitore e posizione mediana degli elettori;
5. confrontare il risultato con il caso di profili totalmente casuali.

Domande guida

- quali regole selezionano candidati più centrali?
- il rumore nelle utilità modifica molto il risultato?
- le regole che premiano il consenso diffuso risultano più stabili?

Output richiesto

- codice sorgente;
- rappresentazioni grafiche della configurazione spaziale;
- statistiche sulle distanze dei vincitori;
- commento comparativo.

11.4 Laboratorio 4 - Blocchi e polarizzazione

Obiettivo

Studiare come la presenza di fazioni elettorali modifichi il comportamento delle diverse regole.

Attività

1. dividere gli elettori in blocchi con preferenze correlate;
2. generare profili con polarizzazione crescente;
3. confrontare pluralità, Borda, runoff e approval;
4. misurare divergenza tra regole e robustezza del vincitore.

Domande guida

- la polarizzazione aumenta la probabilità di esiti divergenti?
- esistono regole più robuste in presenza di elettorati segmentati?
- il candidato di compromesso viene premiato o penalizzato?

Output richiesto

- simulazioni per diversi gradi di polarizzazione;
- tabelle comparative;
- breve discussione sostantiva.

12. Una possibile estensione teorica

Per una lettura piu' avanzata, si puo' introdurre una descrizione probabilistica della distribuzione dei profili.

Nel caso impartial culture, per esempio, il numero di voti di primo posto per ciascun candidato segue una distribuzione multinomiale. Questo permette di collegare:

- il profilo microscopico dei singoli elettori;
- le frequenze aggregate di primi posti;
- la probabilita' che un certo candidato vinca sotto plurality.

In modo analogo, si puo' studiare il comportamento asintotico di alcune osservabili al crescere di N , mostrando che il laboratorio computazionale puo' dialogare con approssimazioni analitiche.

13. Possibili estensioni del modello

Il progetto base puo' essere ampliato in molte direzioni.

13.1 Turnout casuale

Non tutti gli elettori partecipano. Ogni elettore vota con una certa probabilita', eventualmente dipendente dall'intensita' della preferenza.

13.2 Voto strategico

Gli elettori possono non votare sinceramente, ma modificare il proprio comportamento per evitare l'elezione di candidati poco graditi.

13.3 Candidati endogeni

I candidati possono scegliere la propria posizione in uno spazio delle politiche, anticipando la regola elettorale.

13.4 Informazione incompleta

Gli elettori possono osservare segnali rumorosi sulla qualita' dei candidati o sulle intenzioni di voto aggregate.

13.5 Reti sociali e diffusione delle preferenze

Prima del voto, le preferenze possono evolvere attraverso interazioni sociali o campagne informative.

14. Perché questo è un buon case study per il corso

Questo progetto è particolarmente adatto a un corso di metodi computazionali per modelli stocastici per almeno cinque ragioni.

Primo, introduce una forma di stocasticità diversa da quella dei processi dinamici continui o delle catene di Markov classiche: qui l'incertezza sta nella distribuzione dei profili di preferenza.

Secondo, il problema è naturalmente interdisciplinare. Può essere letto in termini di scienza politica, teoria economica della scelta sociale, teoria delle decisioni, analisi algoritmica e sistemi complessi.

Terzo, la struttura computazionale è molto pulita: si genera un input casuale, si applicano più algoritmi di aggregazione e si confrontano statisticamente gli output.

Quarto, il progetto mostra in modo molto concreto che una regola istituzionale non è solo una procedura neutrale, ma un meccanismo che seleziona e filtra l'informazione contenuta nelle preferenze individuali.

Quinto, il modello si presta molto bene a estensioni progressive, da una versione elementare puramente combinatoria fino a modelli spaziali, strategici e dinamici più sofisticati.

15. Conclusione

Il confronto tra regole di voto su profili casuali è un ottimo caso di studio per mostrare come da preferenze individuali perfettamente definite possano emergere esiti collettivi complessi, instabili o fortemente dipendenti dalla procedura adottata.

Dal punto di vista metodologico, il progetto è molto efficace perché unisce:

- modellizzazione discreta;
- generazione casuale di dati;
- simulazione Monte Carlo;
- confronto tra algoritmi;
- analisi statistica degli esiti;
- interpretazione interdisciplinare.

Il punto concettuale piu' importante e' forse questo: la scelta collettiva non e' contenuta soltanto nelle preferenze individuali, ma anche nella regola con cui tali preferenze vengono aggregate.

16. Bibliografia minima

1. Arrow, K. J. Social Choice and Individual Values.
 2. Sen, A. Collective Choice and Social Welfare.
 3. Austen-Smith, D., and Banks, J. S. Positive Political Theory.
 4. Moulin, H. Axioms of Cooperative Decision Making.
 5. Gehrlein, W. V., and Lepelley, D. Voting Paradoxes and Group Coherence.
 6. Myerson, R. B. Game Theory.
-

Appendice A – Pseudocodice Python quasi eseguibile

Questa appendice raccoglie una possibile implementazione, in stile Python essenziale, delle principali componenti computazionali del progetto. Il codice non e' pensato come libreria ottimizzata, ma come base chiara e leggibile per gli studenti.

Le convenzioni usate sono le seguenti:

- i candidati sono etichettati con interi 0, 1, ..., m-1;
- un profilo elettorale e' rappresentato come una lista di classifiche;
- ogni classifica e' una lista ordinata di candidati, dal piu' preferito al meno preferito;
- in caso di parita', si puo' scegliere se usare un tie-breaking lessicografico oppure casuale.

A.1 Import e strutture di base

```
import random
import math
from collections import Counter, defaultdict
from itertools import combinations

# =====
# Utility di base
# =====

def make_rng(seed=None):
    """Restituisce un generatore pseudo-casuale locale."""
```

```

    return random.Random(seed)

def argmax_with_ties(score_dict):
    """
    Restituisce la lista ordinata delle chiavi con punteggio massimo.
    Esempio: {'a': 3, 'b': 5, 'c': 5} -> ['b', 'c']
    """
    max_score = max(score_dict.values())
    winners = [k for k, v in score_dict.items() if v == max_score]
    return sorted(winners)

def break_ties(winners, rng=None, method="lexicographic"):
    """
    Seleziona un solo vincitore da una lista di co-vincitori.
    method = 'lexicographic' -> prende il minimo indice
    method = 'random' -> sceglie casualmente
    """
    if len(winners) == 1:
        return winners[0]

    if method == "lexicographic":
        return min(winners)

    if method == "random":
        if rng is None:
            rng = make_rng()
        return rng.choice(winners)

    raise ValueError("Metodo di tie-breaking non riconosciuto.")

```

A.2 Rappresentazione del profilo

```

# Un profilo e' una lista di classifiche.
# Ogni classifica e' una lista di lunghezza m, contenente una permutazione dei candidati.

# Esempio con m = 4 candidati:
#
# profile = [
#     [2, 0, 1, 3], # elettore 0: preferisce 2 > 0 > 1 > 3
#     [1, 2, 3, 0], # elettore 1: preferisce 1 > 2 > 3 > 0
#     [0, 1, 2, 3], # elettore 2
# ]

```

A.3 Generazione casuale dei profili

A.3.1 Impartial culture

```
def generate_profile_impartial_culture(N, m, rng=None):
    """
    Genera un profilo casuale in impartial culture:
    ogni elettore estrae una permutazione uniforme dei candidati.
    """
    if rng is None:
        rng = make_rng()

    candidates = list(range(m))
    profile = []

    for _ in range(N):
        ballot = candidates[:]      # copia
        rng.shuffle(ballot)
        profile.append(ballot)

    return profile
```

A.3.2 Modello spaziale in una dimensione

```
def generate_profile_spatial_1d(N, m, sigma=0.0, rng=None):
    """
    Genera un profilo spaziale in una dimensione.

    - Gli elettori hanno posizioni  $x_i$  uniformi in  $[0, 1]$ .
    - I candidati hanno posizioni  $y_a$  uniformi in  $[0, 1]$ .
    - L'utilità è data da:
      
$$u_i(a) = -|x_i - y_a| + \text{sigma} * \text{noise}$$


    Restituisce:
    - profile: lista di classifiche
    - voter_positions: lista delle posizioni degli elettori
    - candidate_positions: lista delle posizioni dei candidati
    """
    if rng is None:
        rng = make_rng()

    voter_positions = [rng.random() for _ in range(N)]
    candidate_positions = [rng.random() for _ in range(m)]

    profile = []
    for x in voter_positions:
        utilities = []
```



```

    for a, y in enumerate(candidate_positions):
        noise = sigma * rng.gauss(0.0, 1.0)
        u = -abs(x - y) + noise
        utilities.append((a, u))

    ballot = [a for a, _ in sorted(utilities, key=lambda z: z[1], reverse=True)]
    profile.append(ballot)

    return profile, voter_positions, candidate_positions

```

A.3.3 Profili con blocchi o fazioni

```

def kendall_like_perturbation(base_order, swaps, rng=None):
    """
    Perturba una classifica applicando un certo numero di scambi adiacenti casuali.
    Non e' un campionamento esatto da una distribuzione standard,
    ma e' una procedura semplice e didattica.
    """
    if rng is None:
        rng = make_rng()

    ballot = base_order[:]
    m = len(ballot)

    for _ in range(swaps):
        j = rng.randint(0, m - 2)
        ballot[j], ballot[j + 1] = ballot[j + 1], ballot[j]

    return ballot

def generate_profile_blocks(N, m, block_sizes, swaps_within_block=1, rng=None):
    """
    Genera un profilo con blocchi di elettori.

    Parametri:
    - block_sizes: lista di interi che deve sommare a N
    - swaps_within_block: numero di scambi adiacenti che perturbano
      l'ordine centrale del blocco
    """
    if rng is None:
        rng = make_rng()

    if sum(block_sizes) != N:
        raise ValueError("La somma di block_sizes deve essere N.")

```

```

# Un ordine centrale per ciascun blocco
base_orders = []
for _ in block_sizes:
    order = list(range(m))
    rng.shuffle(order)
    base_orders.append(order)

profile = []
for block_id, size in enumerate(block_sizes):
    base = base_orders[block_id]
    for _ in range(size):
        ballot = kendall_like_perturbation(base, swaps_within_block, rng=rng)
        profile.append(ballot)

rng.shuffle(profile)
return profile

```

A.4 Funzioni di supporto

```

def ranking_to_position_map(ballot):
    """
    Dato un ballot come lista ordinata di candidati,
    restituisce un dizionario candidato -> posizione.
    """
    return {candidate: rank for rank, candidate in enumerate(ballot)}

def profile_to_position_maps(profile):
    """
    Restituisce la lista delle mappe candidato -> posizione
    per tutte le schede del profilo.
    """
    return [ranking_to_position_map(ballot) for ballot in profile]

```

A.5 Regole di voto

A.5.1 Plurality

```

def plurality_scores(profile, m):
    scores = {a: 0 for a in range(m)}
    for ballot in profile:
        scores[ballot[0]] += 1
    return scores

```

```
def plurality_winners(profile, m):
    return argmax_with_ties(plurality_scores(profile, m))
```

A.5.2 Borda count

```
def borda_scores(profile, m):
    scores = {a: 0 for a in range(m)}
    for ballot in profile:
        for rank, candidate in enumerate(ballot):
            scores[candidate] += (m - 1 - rank)
    return scores

def borda_winners(profile, m):
    return argmax_with_ties(borda_scores(profile, m))
```

A.5.3 Approval voting

```
def approval_scores(profile, m, k):
    """
    Ogni elettore approva i primi k candidati della propria classifica.
    """
    if not (1 <= k < m + 1):
        raise ValueError("k deve soddisfare 1 <= k <= m")

    scores = {a: 0 for a in range(m)}
    for ballot in profile:
        approved = ballot[:k]
        for candidate in approved:
            scores[candidate] += 1
    return scores

def approval_winners(profile, m, k):
    return argmax_with_ties(approval_scores(profile, m, k))
```

A.5.4 Confronti testa a testa

```
def pairwise_matrix(profile, m):
    """
    Restituisce una matrice M tale che M[a][b]
    e' il numero di elettori che preferiscono a a b.
    """
    M = [[0 for _ in range(m)] for _ in range(m)]
    position_maps = profile_to_position_maps(profile)

    for pos in position_maps:
```

```

        for a in range(m):
            for b in range(m):
                if a == b:
                    continue
                if pos[a] < pos[b]:
                    M[a][b] += 1

    return M

def pairwise_winner(a, b, M):
    """
    Restituisce:
    - a se a batte b
    - b se b batte a
    - None in caso di pareggio
    """
    if M[a][b] > M[b][a]:
        return a
    elif M[b][a] > M[a][b]:
        return b
    else:
        return None

```

A.5.5 Majority runoff

```

def majority_runoff_winners(profile, m):
    """
    Procedura:
    1. seleziona i primi due candidati per plurality;
    2. tra i due, usa il confronto testa a testa.

    In caso di parita' nei primi due posti plurality,
    questa implementazione usa tutti i co-vincitori possibili
    e poi risolve con una regola semplice.
    """
    pl_scores = plurality_scores(profile, m)
    ordered = sorted(pl_scores.items(), key=lambda z: (-z[1], z[0]))

    # Individuiamo i due finalisti
    top_score = ordered[0][1]
    first_group = [a for a, s in ordered if s == top_score]

    if len(first_group) >= 2:
        finalists = sorted(first_group)[:2]
    else:

```

```

    first = ordered[0][0]
    second_score = ordered[1][1]
    second_group = [a for a, s in ordered if s == second_score]
    second = min(second_group)
    finalists = [first, second]

    a, b = finalists
    M = pairwise_matrix(profile, m)
    head_to_head = pairwise_winner(a, b, M)

    if head_to_head is None:
        return sorted(finalists)

    return [head_to_head]

```

A.5.6 Vincitore di Condorcet

```

def condorcet_winners(profile, m):
    """
    Restituisce la lista dei vincitori di Condorcet.
    In genere e' vuota oppure contiene un solo candidato.
    """
    M = pairwise_matrix(profile, m)
    winners = []

    for a in range(m):
        beats_all = True
        for b in range(m):
            if a == b:
                continue
            if M[a][b] <= M[b][a]:
                beats_all = False
                break
        if beats_all:
            winners.append(a)

    return winners

```

A.6 Rilevazione di cicli di Condorcet

```

def strict_majority_graph(profile, m):
    """
    Costruisce il grafo orientato della maggioranza stretta:
    edge a -> b se a batte b a maggioranza.
    """
    M = pairwise_matrix(profile, m)

```

```

graph = {a: [] for a in range(m)}

for a in range(m):
    for b in range(m):
        if a == b:
            continue
        if M[a][b] > M[b][a]:
            graph[a].append(b)

return graph

def has_directed_cycle(graph):
    """
    Test standard di presenza di ciclo in un grafo orientato
    tramite DFS con colori.
    """
    WHITE = 0
    GRAY = 1
    BLACK = 2

    colour = {node: WHITE for node in graph}

    def dfs(u):
        colour[u] = GRAY
        for v in graph[u]:
            if colour[v] == GRAY:
                return True
            if colour[v] == WHITE and dfs(v):
                return True
        colour[u] = BLACK
        return False

    for node in graph:
        if colour[node] == WHITE:
            if dfs(node):
                return True

    return False

def has_condorcet_cycle(profile, m):
    """
    Restituisce True se il grafo di maggioranza stretta contiene un ciclo.
    """
    graph = strict_majority_graph(profile, m)

```

```
return has_directed_cycle(graph)
```

A.7 Misure di confronto tra regole

```
def same_winner_set(winners1, winners2):
    return set(winners1) == set(winners2)

def rule_outcomes(profile, m, approval_k=2):
    """
    Restituisce tutti gli esiti principali in forma compatta.
    """
    outcomes = {
        "plurality": plurality_winners(profile, m),
        "borda": borda_winners(profile, m),
        "runoff": majority_runoff_winners(profile, m),
        "approval": approval_winners(profile, m, approval_k),
        "condorcet": condorcet_winners(profile, m),
    }
    outcomes["has_cycle"] = has_condorcet_cycle(profile, m)
    return outcomes
```

A.8 Robustezza locale del vincitore

```
def perturb_profile(profile, fraction=0.05, swaps_per_ballot=1, rng=None):
    """
    Modifica una piccola frazione di schede scambiando candidati adiacenti.
    """
    if rng is None:
        rng = make_rng()

    N = len(profile)
    new_profile = [ballot[:] for ballot in profile]
    num_to_change = max(1, int(fraction * N))

    chosen_voters = rng.sample(range(N), num_to_change)

    for i in chosen_voters:
        ballot = new_profile[i]
        m = len(ballot)
        for _ in range(swaps_per_ballot):
            j = rng.randint(0, m - 2)
            ballot[j], ballot[j + 1] = ballot[j + 1], ballot[j]

    return new_profile
```

```

def empirical_robustness(profile, m, rule_func, trials=100, fraction=0.05, rng=None):
    """
    Stima la probabilita' che il vincitore non cambi sotto piccole perturbazioni.
    rule_func deve restituire la lista dei co-vincitori.
    """
    if rng is None:
        rng = make_rng()

    base_winners = set(rule_func(profile, m))
    stable = 0

    for _ in range(trials):
        perturbed = perturb_profile(profile, fraction=fraction, rng=rng)
        new_winners = set(rule_func(perturbed, m))
        if new_winners == base_winners:
            stable += 1

    return stable / trials

```

A.9 Distanza del vincitore dal votante mediano nel modello spaziale

```

def median_position(values):
    """
    Restituisce la mediana di una lista di valori.
    """
    xs = sorted(values)
    n = len(xs)
    mid = n // 2

    if n % 2 == 1:
        return xs[mid]
    else:
        return 0.5 * (xs[mid - 1] + xs[mid])

def distance_winner_to_median(winners, candidate_positions, voter_positions):
    """
    Se ci sono piu' co-vincitori, restituisce la distanza media
    dei co-vincitori dal votante mediano.
    """
    x_med = median_position(voter_positions)
    distances = [abs(candidate_positions[a] - x_med) for a in winners]
    return sum(distances) / len(distances)

```


A.10 Simulazione Monte Carlo generale

```
def monte_carlo_experiment(
    N,
    m,
    S,
    profile_model="ic",
    approval_k=2,
    sigma=0.0,
    block_sizes=None,
    swaps_within_block=1,
    seed=None,
):
    """
    Esegue S simulazioni Monte Carlo e restituisce un dizionario
    con alcune statistiche aggregate.
    """
    rng = make_rng(seed)

    stats = {
        "plurality_vs_borda_diff": 0,
        "plurality_vs_runoff_diff": 0,
        "borda_vs_approval_diff": 0,
        "condorcet_exists": 0,
        "condorcet_cycle": 0,
        "plurality_winner_counts": Counter(),
        "borda_winner_counts": Counter(),
        "runoff_winner_counts": Counter(),
        "approval_winner_counts": Counter(),
    }

    for _ in range(S):
        if profile_model == "ic":
            profile = generate_profile_impartial_culture(N, m, rng=rng)

        elif profile_model == "spatial":
            profile, voter_positions, candidate_positions = generate_profile_spatial_1d(
                N, m, sigma=sigma, rng=rng
            )

        elif profile_model == "blocks":
            if block_sizes is None:
                raise ValueError("Per profile_model='blocks' occorre specificare block_sizes")
            profile = generate_profile_blocks(
                N, m, block_sizes=block_sizes,
                swaps_within_block=swaps_within_block,
```

```

        rng=rng
    )

    else:
        raise ValueError("Modello di profilo non riconosciuto.")

    outcomes = rule_outcomes(profile, m, approval_k=approval_k)

    pl = tuple(outcomes["plurality"])
    bo = tuple(outcomes["borda"])
    ru = tuple(outcomes["runoff"])
    ap = tuple(outcomes["approval"])
    co = tuple(outcomes["condorcet"])

    stats["plurality_winner_counts"][pl] += 1
    stats["borda_winner_counts"][bo] += 1
    stats["runoff_winner_counts"][ru] += 1
    stats["approval_winner_counts"][ap] += 1

    if set(pl) != set(bo):
        stats["plurality_vs_borda_diff"] += 1

    if set(pl) != set(ru):
        stats["plurality_vs_runoff_diff"] += 1

    if set(bo) != set(ap):
        stats["borda_vs_approval_diff"] += 1

    if len(co) > 0:
        stats["condorcet_exists"] += 1

    if outcomes["has_cycle"]:
        stats["condorcet_cycle"] += 1

    # normalizzazione
    results = {
        "P_diff_plurality_borda": stats["plurality_vs_borda_diff"] / S,
        "P_diff_plurality_runoff": stats["plurality_vs_runoff_diff"] / S,
        "P_diff_borda_approval": stats["borda_vs_approval_diff"] / S,
        "P_condorcet_exists": stats["condorcet_exists"] / S,
        "P_condorcet_cycle": stats["condorcet_cycle"] / S,
        "plurality_winner_counts": stats["plurality_winner_counts"],
        "borda_winner_counts": stats["borda_winner_counts"],
        "runoff_winner_counts": stats["runoff_winner_counts"],
        "approval_winner_counts": stats["approval_winner_counts"],
    }
}

```

```
return results
```

A.11 Esempio minimo di utilizzo

```
if __name__ == "__main__":
    N = 100
    m = 4
    S = 1000
    approval_k = 2

    results = monte_carlo_experiment(
        N=N,
        m=m,
        S=S,
        profile_model="ic",
        approval_k=approval_k,
        seed=123
    )

    print("Probabilita' che plurality e Borda diano esiti diversi:")
    print(results["P_diff_plurality_borda"])

    print("\nProbabilita' di esistenza di un vincitore di Condorcet:")
    print(results["P_condorcet_exists"])

    print("\nProbabilita' di ciclo di Condorcet:")
    print(results["P_condorcet_cycle"])
```

A.12 Esempio per il caso spaziale

```
if __name__ == "__main__":
    rng = make_rng(321)

    N = 101
    m = 5
    sigma = 0.10

    profile, voter_positions, candidate_positions = generate_profile_spatial_1d(
        N=N, m=m, sigma=sigma, rng=rng
    )

    pl = plurality_winners(profile, m)
    bo = borda_winners(profile, m)
    ru = majority_runoff_winners(profile, m)
```

```

print("Posizioni candidati:", candidate_positions)
print("Vincitore plurality:", pl)
print("Vincitore Borda:", bo)
print("Vincitore runoff:", ru)

print("Distanza plurality dal mediano:",
      distance_winner_to_median(pl, candidate_positions, voter_positions))
print("Distanza Borda dal mediano:",
      distance_winner_to_median(bo, candidate_positions, voter_positions))

```

A.13 Possibili esercizi computazionali immediati

1. Stimare, in impartial culture, la probabilit  che plurality e Borda selezionino vincitori diversi al variare di N e m .
2. Stimare la frequenza di cicli di Condorcet per $m = 3, 4, 5$.
3. Studiare come cambia il risultato di approval voting al variare di k .
4. Nel modello spaziale unidimensionale, confrontare la distanza del vincitore dal votante mediano sotto plurality, Borda e runoff.
5. Introdurre blocchi elettorali e verificare se la polarizzazione aumenta la divergenza tra regole.
6. Stimare la robustezza locale del vincitore sotto piccole perturbazioni del profilo.

A.14 Nota metodologica

Il codice qui riportato   volutamente semplice e quasi minimale. In una versione piu' avanzata si potrebbero aggiungere:

- visualizzazioni grafiche;
- gestione sistematica delle parita';
- classi Python per separare profili, regole di voto e osservabili;
- ottimizzazioni per numeri grandi di candidati;
- confronto con risultati teorici noti per impartial culture.